



STATION MÉTÉO AGRICOLE

Documentation Technique Complète

Plateforme IoT de Surveillance Climatique Agricole — Sénégal

Université	USSEIN — Université du Sine Saloum El Hâdj Ibrahima Niass
Année	2025 – 2026
Type	Projet de Mémoire de fin d'études — IoT Agricole
Stack	Python · FastAPI · Streamlit · Firebase · Scikit-learn
Matériel	ESP32 + SIM7600 · Capteurs DHT22 · Anémomètre

1. Architecture Globale du Système

Le système est composé de 4 couches interconnectées qui permettent de collecter les données climatiques sur le terrain, les stocker dans le cloud, les traiter et les afficher à l'agriculteur en temps réel.

Schéma 1 — Architecture globale : ESP32 → Firebase → FastAPI → Streamlit

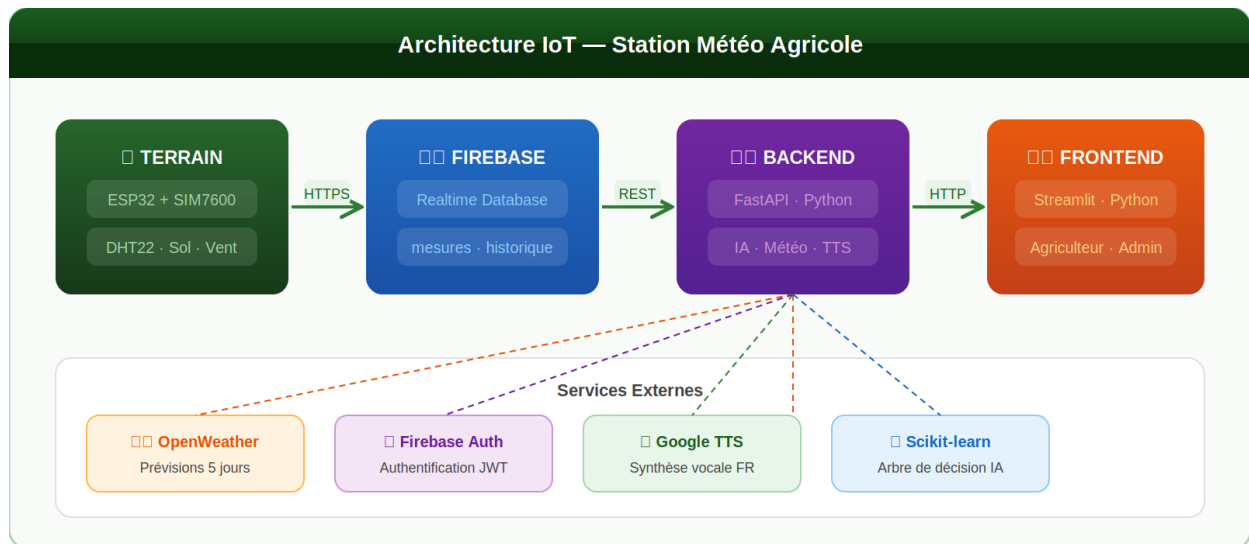


Figure 1 : Vue d'ensemble de l'architecture IoT à 4 couches.

Couche	Rôle
Couche 1 — Terrain	ESP32 avec SIM7600 lit les capteurs (DHT22, humidité sol, anémomètre) et envoie les données via HTTPS à Firebase toutes les X minutes.
Couche 2 — Cloud	Firebase Realtime Database reçoit et stocke les mesures. Accessible depuis n'importe où. Structure hiérarchique JSON.
Couche 3 — Backend	API FastAPI orchestre les services : lecture Firebase, modèle IA, prévisions météo OpenWeather, synthèse vocale gTTS.
Couche 4 — Frontend	Application Streamlit affiche en temps réel : métriques, graphiques Plotly, carte Folium, recommandations IA.

1.1 Structure de la base de données Firebase

Toutes les données sont organisées dans Firebase Realtime Database selon la structure suivante :

Schéma 2 — Structure Firebase Realtime Database

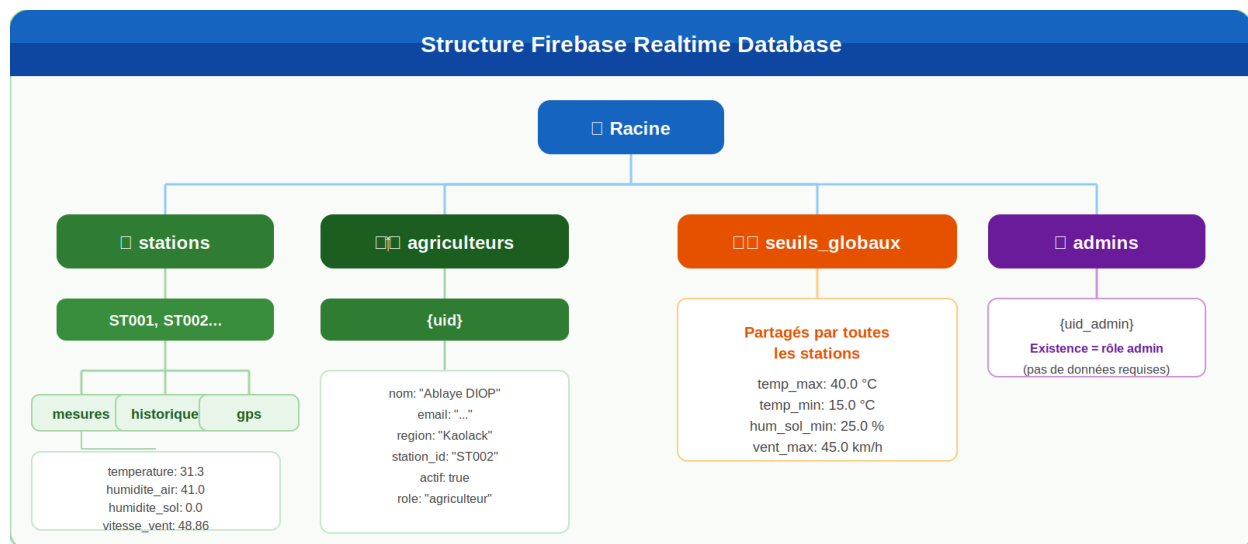


Figure 2 : Nœuds Firebase et leurs contenus.

Chemin Firebase	Contenu
stations/{id}/mesures	Mesures actuelles : temperature, humidite_air, humidite_sol, vitesse_vent, timestamp
stations/{id}/historique	Historique complet des mesures (dict avec clés auto-générées par Firebase)
stations/{id}/gps	Coordonnées GPS de la station : latitude, longitude
agriculteurs/{uid}	Profil agriculteur : nom, email, region, station_id, actif, role, date_creation
admins/{uid}	Existence du nœud = rôle administrateur (pas de données requises)
seuils_globaux	Seuils partagés par toutes les stations : temp_max, temp_min, hum_sol_min, vent_max

2. Flux de Données Complet

Ce schéma montre le chemin complet d'une donnée : depuis la lecture par l'ESP32 sur le terrain jusqu'à l'affichage sur l'écran de l'agriculteur, avec tous les services impliqués.

Schéma 3 — Flux de données temps réel bout en bout

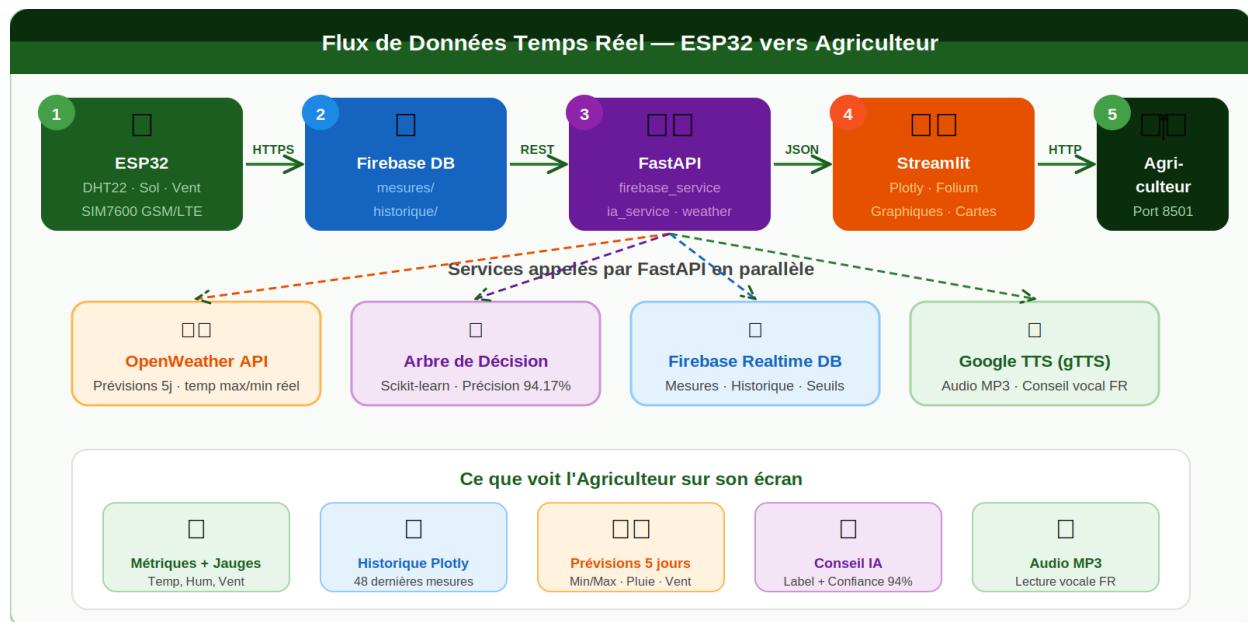


Figure 3 : Flux complet en 5 étapes avec services parallèles.

#	Étape	Détail
1	ESP32 lit les capteurs	DHT22 (temp + hum_air), capteur sol (hum_sol), anémomètre (vent). Envoi HTTPS via SIM7600 vers Firebase.
2	Firebase stocke	stations/ST002/mesures mis à jour. Entrée ajoutée dans stations/ST002/historique.
3	Frontend interroge FastAPI	GET /mesures/ST002, GET /historique, GET /previsions, GET /seuils en parallèle.
4	FastAPI traite	Lit Firebase, appelle OpenWeather (météo), exécute le modèle IA, génère l'audio TTS.
5	Streamlit affiche	Métriques, jauges Plotly, graphiques historique, cartes prévisions météo, conseil IA + bouton audio.

3. Système d'Authentification

Le système distingue deux rôles : Administrateur et Agriculteur. L'authentification passe par Firebase Auth (JWT) puis par une vérification du rôle dans Firebase DB.

Schéma 4 — Flux d'authentification et de connexion

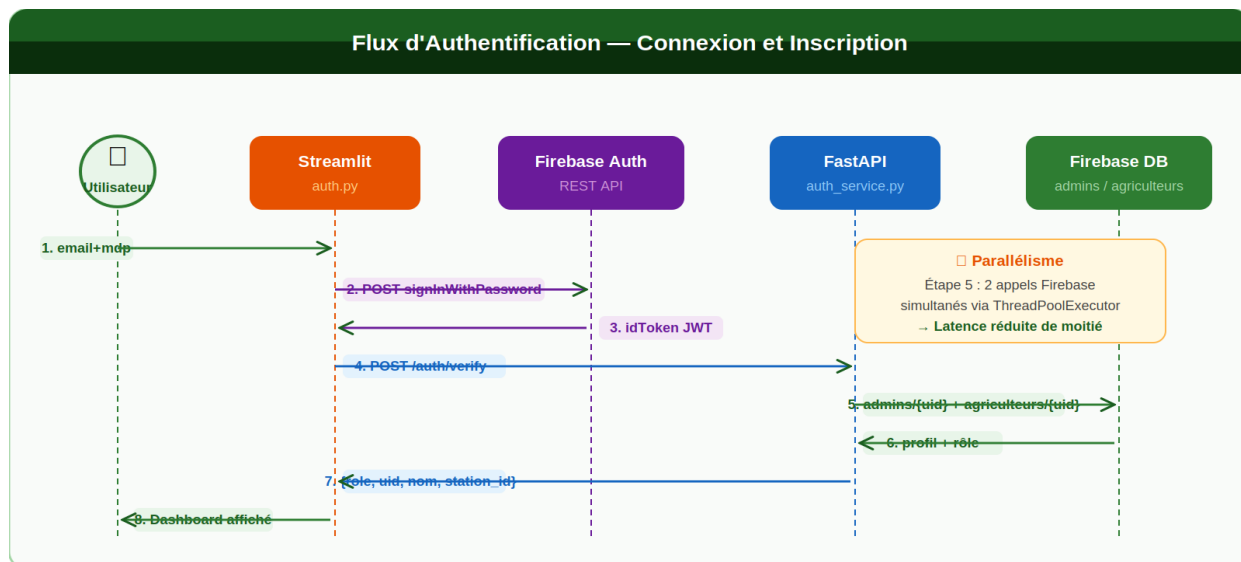


Figure 4 : Séquence de connexion avec parallélisme Firebase (ThreadPoolExecutor).

3.1 Fichiers concernés

Fichier	Rôle
frontend/components/auth.py	Page login + inscription. Switcher  /  . Appelle Firebase REST et le backend.
backend/auth_service.py	Vérifie le token JWT, interroge Firebase DB en parallèle pour déterminer le rôle.
backend/firebase_service.py	verify_id_token() : décode le JWT manuellement (sans SDK) pour éviter les erreurs d'horloge.

3.2 Pourquoi le décodage JWT manuel ?

Le SDK Firebase Admin vérifie l'horloge système. Si elle est décalée de plus de 60 secondes par rapport aux serveurs Firebase, il rejette le token. Le décodage manuel de la partie centrale du JWT (base64) contourne ce problème. La sécurité reste garantie car Firebase Auth a déjà validé email + mot de passe.

3.3 Optimisation par parallélisme

Lors de la vérification du rôle, deux appels Firebase sont nécessaires : `admins/{uid}` et `agriculteurs/{uid}`. Sans parallélisme, ces appels sont séquentiels (2x latence Firebase). Avec `ThreadPoolExecutor`, les deux appels se font simultanément, divisant la latence par deux.

```
with concurrent.futures.ThreadPoolExecutor(max_workers=2) as executor:
```



```
f_admin = executor.submit(lambda: db.reference(f'admins/{uid}').get())
f_agri   = executor.submit(lambda: db.reference(f'agriculteurs/{uid}').get())
admin_data = f_admin.result(timeout=20)
agri_data  = f_agri.result(timeout=20)
```

4. Backend — API FastAPI (Port 8000)

Le backend est une API REST développée avec FastAPI. Il est le cerveau du système : il lit Firebase, exécute le modèle IA, récupère la météo et génère la voix.

4.1 Routes API disponibles

Méthode	Route	Description	Port
GET	GET /	Vérification santé de l'API	8000
POST	POST /auth/verify	Vérifie le token Firebase → retourne rôle + profil	8000
POST	POST /auth/register	Inscrit un nouvel agriculteur dans Firebase DB	8000
GET	GET /mesures/{id}	Mesures actuelles d'une station	8000
GET	GET /historique/{id}	Historique des N dernières mesures	8000
GET	GET /previsions/{id}	Prévisions météo 5 jours (OpenWeather)	8000
POST	POST /recommandation	Recommandation IA basée sur les 4 mesures	8000
POST	POST /tts	Synthèse vocale MP3 depuis texte en français	8000
GET	GET /stations	Toutes les stations avec mesures (admin)	8000
GET	GET /agriculteurs	Liste de tous les agriculteurs	8000
GET	GET /seuils	Seuils d'alerte globaux	8000
POST	POST /seuils	Mise à jour des seuils globaux	8000
GET	GET /debug/{id}	Diagnostic structure Firebase (développement)	8000

4.2 firebase_service.py — Lecture Firebase

Ce module gère toutes les interactions avec Firebase. Il est conçu pour être flexible sur les noms de champs envoyés par l'ESP32 :

Champ standard	Variante acceptées (ESP32)
temperature	Accepte : temperature, temp, Temp, TEMP, temperature_c
humidite_air	Accepte : humidite_air, humidite, humidity, hum_air, hum
humidite_sol	Accepte : humidite_sol, sol, soil, hum_sol, moisture
vitesse_vent	Accepte : vitesse_vent, vent, wind, wind_speed, vitesseVent
timestamp	Accepte : timestamp, time, date, ts

4.3 weather_service.py — Prévisions Météo

Interroge l'API OpenWeather /forecast avec cnt=40 (40 entrées de 3h = 5 jours). Calcule le vrai max/min sur toute la journée en parcourant toutes les entrées. Utilise la description de 12h00 comme référence.

```
# Regroupement par jour et calcul max/min réel
jours[jour]['temp_max'] = max(jours[jour]['temp_max'], item['main']['temp_max'])
jours[jour]['temp_min'] = min(jours[jour]['temp_min'], item['main']['temp_min'])
if '12:00:00' in item['dt_txt']: # Description de midi = plus représentative
    jours[jour]['description'] = item['weather'][0]['description']
```

4.4 tts_service.py — Synthèse Vocale

Utilise gTTS (Google Text-to-Speech) pour convertir le conseil IA en audio MP3 en français. L'audio est généré en mémoire (BytesIO) sans créer de fichier sur le disque.

```
tts = gTTS(text=texte, lang='fr', slow=lent)
buffer = io.BytesIO()
tts.write_to_fp(buffer) # En mémoire, pas de fichier disque
return buffer.read()   # Bytes MP3 retournés directement à Streamlit
```


5. Modèle IA — Arbre de Décision Agricole

Le système de recommandation utilise un Decision Tree Classifier (scikit-learn) entraîné sur 3 000 exemples synthétiques représentatifs du climat sénégalais.

Schéma 5 — Arbre de Décision (règles de recommandation)

L'arbre teste les 4 valeurs capteurs dans l'ordre de priorité suivant :

Priorité	Condition	Recommandation
Priorité 1	temp > 38°C ET hum_sol < 30%	🚨 URGENCE — Stress hydrique sévère — Irriguer immédiatement
Priorité 2	vent > 42 km/h	🌪️ Vent fort — Protéger les cultures, reporter les traitements
Priorité 3	hum_air > 75% ET temp > 28°C	🍄 Risque fongique — Traitement préventif recommandé
Priorité 4	hum_sol < 35%	💧 Sol sec — Arroser, humidité insuffisante pour les cultures
Priorité 5	Aucune condition critique	✅ Conditions favorables — Surveiller et maintenir

5.1 Paramètres du modèle

Algorithme	DecisionTreeClassifier (scikit-learn)
Profondeur max	max_depth = 7 niveaux
Dataset	3 000 exemples synthétiques + 5% de bruit aléatoire
Split	80% entraînement / 20% test
Précision	94.17% sur le jeu de test
Entrée	4 features : temperature, humidite_air, humidite_sol, vitesse_vent
Sortie	Classe (0-4) + probabilités → label + emoji + conseil + score de confiance

5.2 Exemple de prédiction

```
# Entrée capteurs
X = [[31.3, 41.0, 0.0, 48.86]] # temp, hum_air, hum_sol, vent

# Prédiction
classe = modele.predict(X)[0] # Ex: 1 (vent fort)
confiance = modele.predict_proba(X)[0][1] # Ex: 0.94

# Sortie retournée au frontend
{ 'label': 'Vent fort', 'emoji': '🌪️',
  'conseil': 'Protéger les cultures...', 'confiance': 0.94 }
```

6. Frontend — Application Streamlit (Port 8501)

L'interface est développée avec Streamlit. Elle applique un thème vert très foncé avec fond photo de champ agricole, inspiré du projet `Station_meteo`.

6.1 Routage selon le rôle

Non connecté	→ <code>page_login()</code> : formulaire connexion + inscription (switcher 2 onglets)
role = 'admin'	→ <code>page_admin()</code> : tableau de bord global avec toutes les stations
role = 'agriculteur'	→ <code>pageagriculteur()</code> : dashboard personnel lié à une station

6.2 Dashboard Agriculteur — Ordre des sections

#	Section	Contenu
1	 Mesures temps réel	4 métriques Streamlit : Température, Humidité air, Humidité sol, Vent + delta alerte
2	 Niveaux actuels	4 jauges Plotly circulaires avec seuils colorés (rouge si dépassement)
3	 Historique	5 onglets graphiques Plotly (courbe + aire remplie) pour chaque capteur
4	 Prévisions météo	5 cartes CSS (5 jours) : emoji, temp min→max, description, humidité, vent
5	 Conseil IA	Carte recommandation : emoji, label, conseil détaillé, confiance en %
6	Footer	Station Météo Agricole · Réseau IoT Sénégal · USSEIN

6.3 Sidebar de navigation

Avatar utilisateur	Emoji  (agriculteur) ou  (admin), nom, rôle, station
Navigation radio	Agriculteur : Accueil / Historique / Prévisions Admin : 4 onglets
Sélecteur région	14 régions sénégalaises — met à jour les prévisions météo
Actualisation auto	Checkbox — <code>time.sleep(30) + st.rerun()</code> si activé
Résumé station	ID, région, stack technique
Déconnexion	Bouton rouge discret — supprime toute la session Streamlit

6.4 Composants graphiques (`charts.py`)

<code>graphique_historique()</code>	Courbe Plotly avec aire remplie. Parse timestamps FR ('dd/mm/YYYY à HH:MM') et ISO.
<code>graphique_jauge()</code>	Jauge circulaire Plotly. Rouge si > <code>seuil_max</code> , orange si < <code>seuil_min</code> .

<code>graphique_tous_capteurs()</code>	Vue globale multi-lignes avec normalisation 0-100% pour comparer les 4 capteurs.
<code>_hex_to_rgba()</code>	Convertit #RRGGBB → rgba(R,G,B,alpha). Nécessaire car Plotly n'accepte pas les hex 8 caractères.

7. Design System — Thème Visuel

Le fichier `css/styles.css` définit l'ensemble du thème inspiré de `Station_meteo`. Le fond est une photo de champ agricole avec superposition vert foncé.

7.1 Palette de couleurs

Variable CSS	Valeur	Usage
<code>--v-noir</code>	<code>#0A2E0C</code>	Vert quasi-noir — sidebar, entêtes
<code>--v-foret</code>	<code>#1B5E20</code>	Vert forêt — bordures top des cartes métriques
<code>--v-fonce</code>	<code>#2E7D32</code>	Vert foncé — titres, accents
<code>--v-vif</code>	<code>#43A047</code>	Vert vif — badges actifs, hover, live dot animé
<code>--v-doux</code>	<code>#A5D6A7</code>	Vert doux — bordures légères
<code>--v-pale</code>	<code>#E8F5E9</code>	Fond vert très pâle — KPI, badges ok
<code>--rouge</code>	<code>#B71C1C</code>	Rouge — alertes critiques, bouton déconnexion
<code>--orange</code>	<code>#E65100</code>	Orange — avertissements, déltas

7.2 Typographie

Police	Poids	Usage
Nunito	400/700/900	Corps de texte, labels — lisible et moderne
Sora	600/800	Titres et entêtes — impactant
Roboto Mono	500	Valeurs numériques dans les métriques et jauges

8. Guide de Déploiement

8.1 Prérequis

Python	3.10 ou supérieur
Packages pip	fastapi uvicorn streamlit firebase-admin scikit-learn numpy pandas plotly folium streamlit-folium gtts requests
serviceAccountKey.json	Téléchargé depuis Firebase Console → Paramètres du projet → Comptes de service
Clé OpenWeather	Inscrivez-vous sur openweathermap.org → clé gratuite dans weather_service.py
Clé Firebase Auth	Firebase Console → Paramètres → Config web → apiKey dans auth.py

8.2 Lancement (2 terminaux simultanés)

Terminal 1 — Backend (Port 8000)

```
cd Agri_Station_meteo/backend && uvicorn main:app --reload
```

Terminal 2 — Frontend (Port 8501)

```
cd Agri_Station_meteo/frontend && streamlit run app.py
```

8.3 URLs utiles

Page	URL	Description
Interface web	http://localhost:8501	Application Streamlit pour agriculteurs et admins
Documentation API	http://localhost:8000/docs	Swagger UI — tester toutes les routes
Diagnostic Firebase	http://localhost:8000/debug/ST002	Voir la structure réelle Firebase

⚠ Important : Le fichier serviceAccountKey.json contient votre clé privée Firebase. Ne le partagez jamais et ajoutez-le à .gitignore.